Fall Hacks 2026
Git & GitHub

Presented By: Cameron, Ben, Gleb



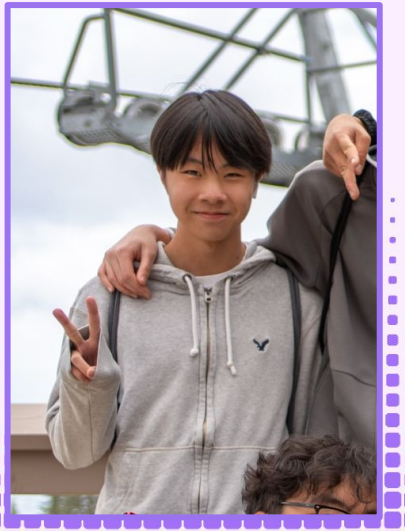
About Us!

Ben



Hey everyone, I'm **Ben**! I'm a first-year CS student at SFU, excited to learn as much as I can, work on cool projects, and meet new people along the way.

Cameron



I'm **Cameron**! I'm a first year Computer Science student aiming to specialize in AI and connect with as many new people as possible along the way.

Gleb



I'm **Gleb**, a computer science that student from Kazakhstan (grew up in UAE, Dubai) that is aspiring to become a quantitative developer

PRE-WORKSHOP TASKS

Choosing your Terminal

MacOS



Terminal - Pre-installed

Shell: Zsh

Mac runs on a Unix foundation, meaning its native **Terminal** speaks in Unix commands! Everything just works out of the box.

Windows



Powershell - Don't use cmd!

Shell: Powershell

Windows has legacy tools like cmd, but **PowerShell** is the modern, powerful command-line shell built into Windows.

Linux



? ? ? - Use whatever is pre-installed

Shell: ? ? ?

Linux is completely modular, so different distributions use different default shells or terminals. If you're on Linux, use your **favorite one!**

PRE-WORKSHOP TASKS

Room Git Setup

Run in the terminal before starting:

```
git config --global user.name "Your Name"
git config --global user.email "xx@example.com"
git config --global pull.rebase false
```

Use your exact GitHub email for commit credit!

Need Help?

Don’t be shy, raise your hand.
We’re happy to help!

MENTAL MODEL

Git vs GitHub?



Git (Local)

Time Machine: Tracks all changes in your code and records it

Offline First: Zero Wi-Fi required.



GitHub (Cloud)

Cloud Backup: Google Drive for your code

Collaboration: Lets others view / interact with your code

Four Hands-On Exercises



Exercise 1

Local commit loop & custom profile README



Exercise 2

Partner template setup & blocking .env secrets



Exercise 3

Feature branches & merging Pull Requests



Exercise 4

Triggering and resolving merge conflicts

EXERCISE 1

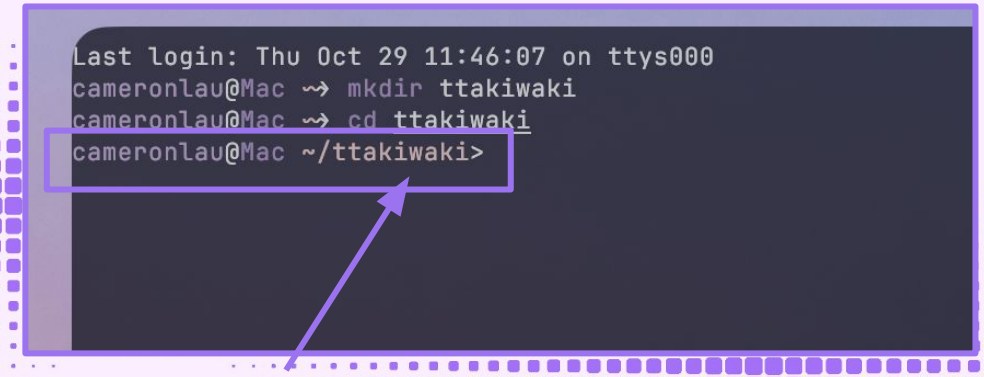
Commit Loop & Special Repo

- Open your projects directory and create a new folder matching your GitHub username exactly.

  You can make it in your IDE, or in the terminal. Run:

```
mkdir <github-username>
cd <github-username>
```

Terminal example:



I'm now in the new directory!

Concept:

`mkdir` - stands for 'make directory'
`cd` - stands for change directory'

These are Unix type commands (Linux & MacOS), but work on windows because of aliases!

EXERCISE 1

Commit Loop & Special Repo

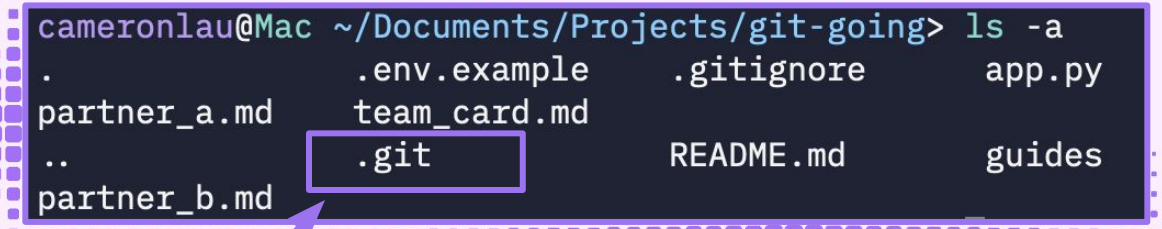
- Using the terminal, initialize the folder as a Git repository. Run:

```
git init
```

- This created a hidden .git folder for you. It keeps track of all changes. Show it using:

```
Mac & Linux: ls -a

Windows: ls -force
```



Concept:

Items starting with . is a **dotfile (hidden file)** used for config and metadata. The .git folder is the time machine. If you delete it, the folder forgets its entire Git history.

EXERCISE 1

Commit Loop & Special Repo

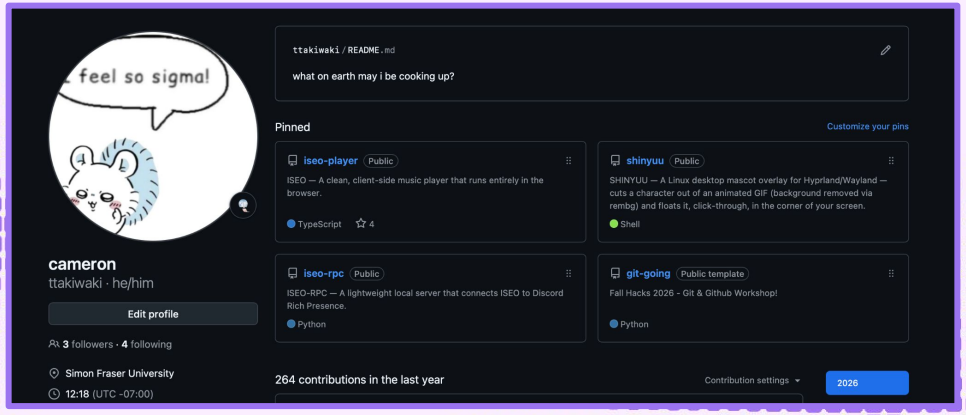
- Start by creating a README.md file in your folder. Create a file in the IDE or by running:

```
Mac & Linux: touch README.md

Windows: ni README.md

code README.md (enter the file for VS Code users using terminal up till now.)
```

- Now Inside write what you'd like displayed on your profile





EXERCISE 1

Commit Loop & Special Repo

- The Commit Loop:

```
git status

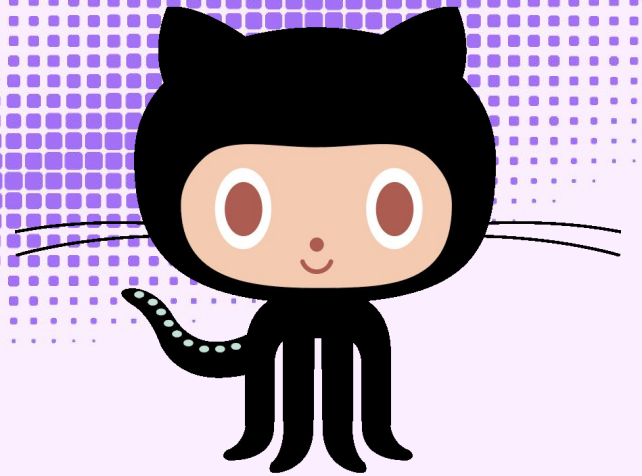
git add README.md

git commit -m “initial commit”

git log --oneline
```

Concept:

`git status` should return README.md in red to indicate that it is untracked / modified. After adding the file it will return green! Git status displays the current state of your working directory and staging area

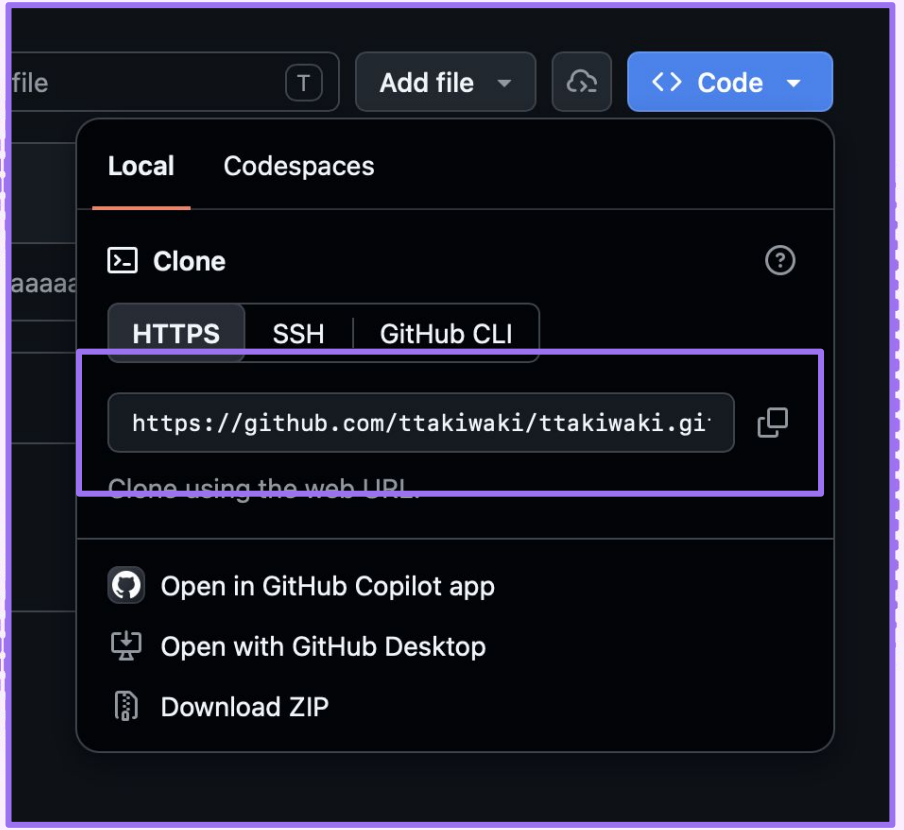


EXERCISE 1

Commit Loop & Special Repo

- Go to www.github.com and create a repository (top right). **Make sure the repository name matches your username!** Make it public and **Don't add a README, we created one already!**

- Get the link to the repository (Code button -> dropdown and copy the HTTPS link.)



EXERCISE 1

Commit Loop & Special Repo

- Back in your IDE run:

```
git remote add origin <URL>

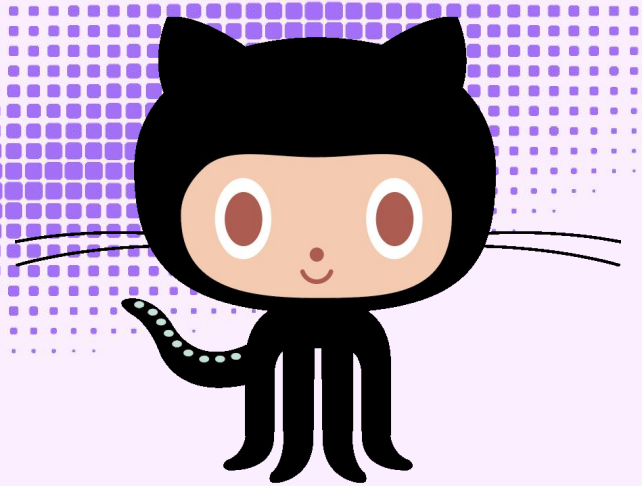
git branch -M main

git push -u origin main
```

- Navigate to your GitHub profile to see the live changes to your profile!

Concept:

`origin` is just a short name that points to a remote Git repository URL. You set it when running the command, so future pushes are easier.



EXERCISE 1

The Commit Loop

Don’t need to run.

git add
Pack files inside the shipping box

git commit
Seal the box with a message

git push
Ship the box to the cloud

```
git add .          # the . means all files in the folder

git commit -m“”    # write a meaningful message to describe changes

git push           # pushes the file to GitHub or other hosting platforms
```

EXERCISE 2

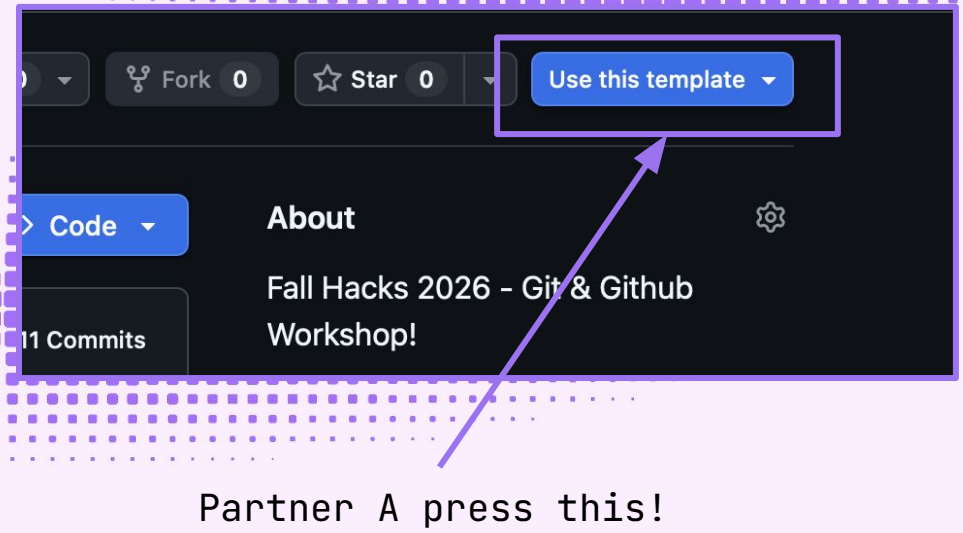
Env Files & Branches

- Find a partner or we can pair you with someone!
- Each partner follow only your specific instructions. **Very Important!**

Partner A

https://github.com/ttakiwaki/git-going

- After clicking the **template button**, leave all the settings as they are, scroll to down and press **Create** at the bottom!



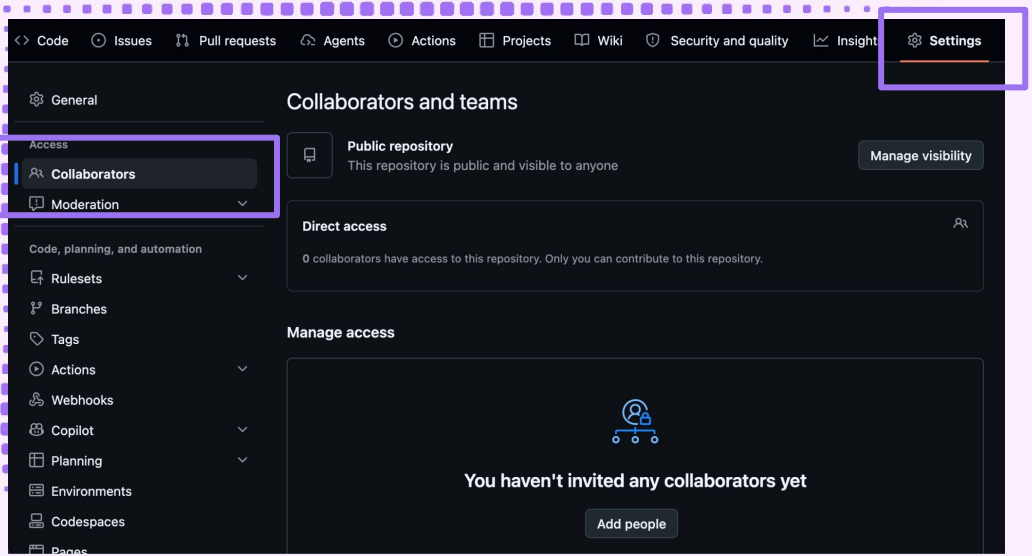
EXERCISE 2

Env Files & Branches

- Find a partner or we can pair you with someone!
- Each partner follow only your specific instructions. **Very Important!**

Partner A

- Under your new repository, navigate to the **Settings Tab -> Collaborators.**
- Add Partner B as one of your collaborators.



Here it is!

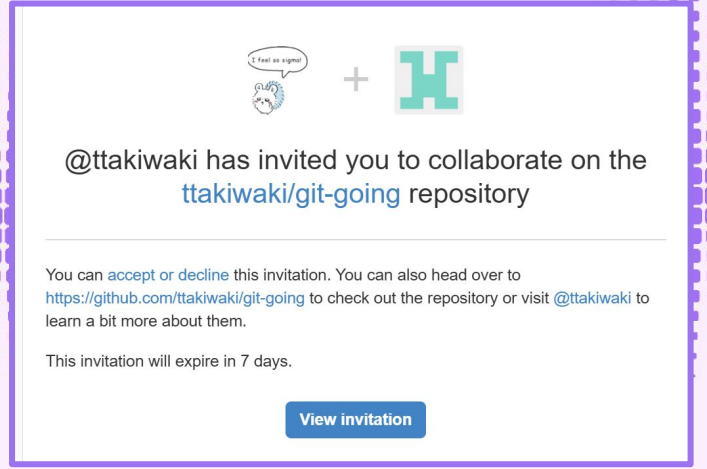
EXERCISE 2

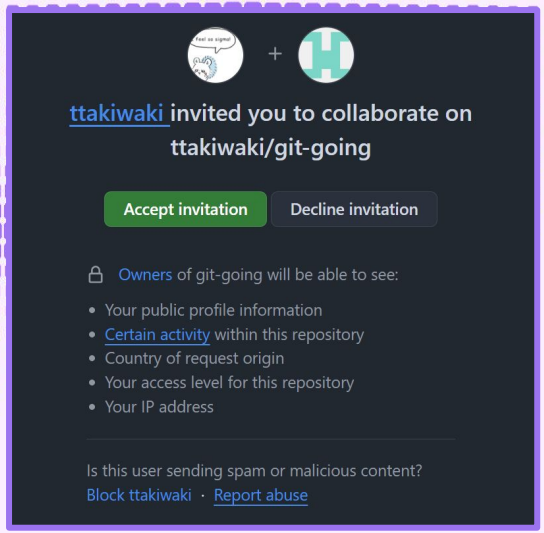
Env Files & Branches

- Find a partner or we can pair you with someone!
- Each partner follow only your specific instructions. **Very Important!**

Partner B

- Check the email used to create your GitHub account, as A's invitation is sent to your email.
- And accept the invitation.





EXERCISE 2

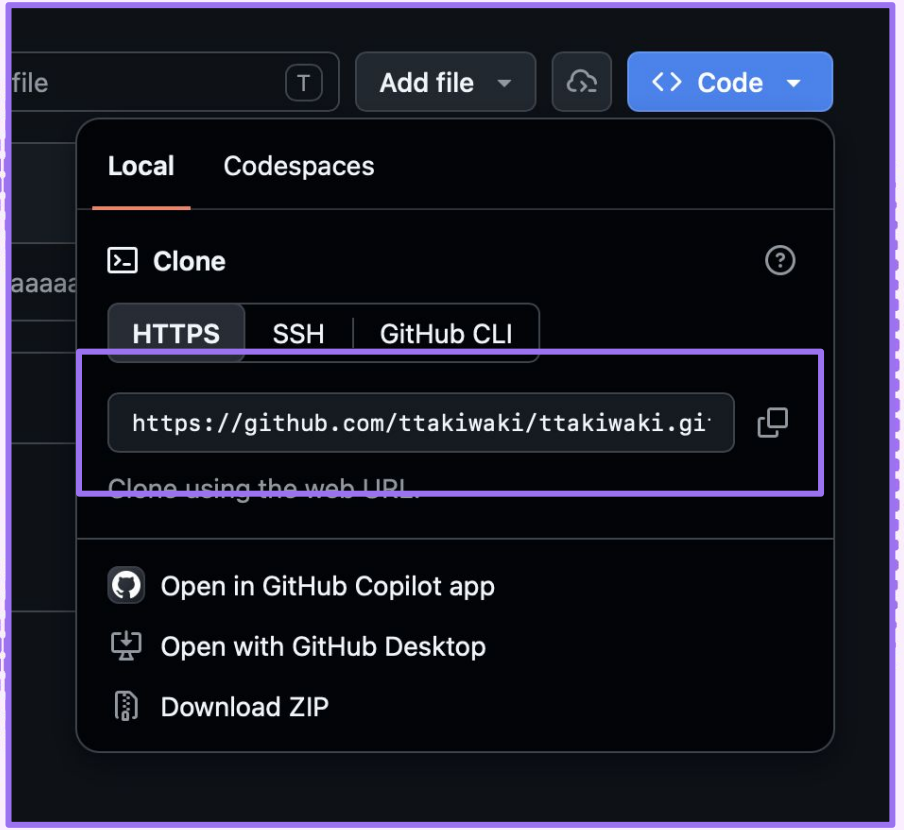
Env Files & Branches

- Partners A & B copy A's repository link under the code dropdown again
- **OPEN A NEW TERMINAL SESSION or** (make sure you're not at /<username>) **and run:**

```
git clone <url>

cd git-going

code .  (enter the file for VS Code users using terminal up till now.)
```



EXERCISE 2

Env Files & Branches

- Inside run the command:

```
cp .env.example .env
```

- A new file should have spawned inside of your codebase. What is a .env file?

- We put all stuff like API keys inside of .env files instead of hardcoding them.

.env

Protect your Sensitive Information!

People use bots to scrape GitHub repositories for sensitive information such as API keys! It would be catastrophic if someone spent $200 on credits using your money.

EXERCISE 2

Env Files & Branches

- How do we access these in our code if they're stored in other files?

- Here we have a Python Example! You can try it yourself if you have Python installed. Add your name to the .env file and try running the code:

`Mac & Linux: python3 app.py`

`Windows: python app.py`

```python
import os


def load_env():
    """Simple helper to read a local .env file without external libraries."""
    if not os.path.exists(".env"):
        print("⚠️  No .env file found! Copy .env.example to .env first.")
        return
    with open(".env") as f:
        for line in f:
            line = line.strip()
            if line and not line.startswith("#") and "=" in line:
                key, value = line.split("=", 1)
                os.environ[key] = value.strip('"').strip("'")


def main():
    load_env()
    name = os.environ.get("NAME", "Unknown")
    secret = os.environ.get("SECRET_API_KEY", None)

    print(f"👋 Welcome to Git Going, {name}!")
    if secret:
        print(f"🔒 Secret loaded locally (First 4 chars: {secret[:4]}****)")
    else:
        print("X No SECRET_API_KEY loaded.")


if __name__ == "__main__":
    main()
```

EXERCISE 2

Env Files & Branches

- If it's so important then how do we protect it? Run:

```
git status
```

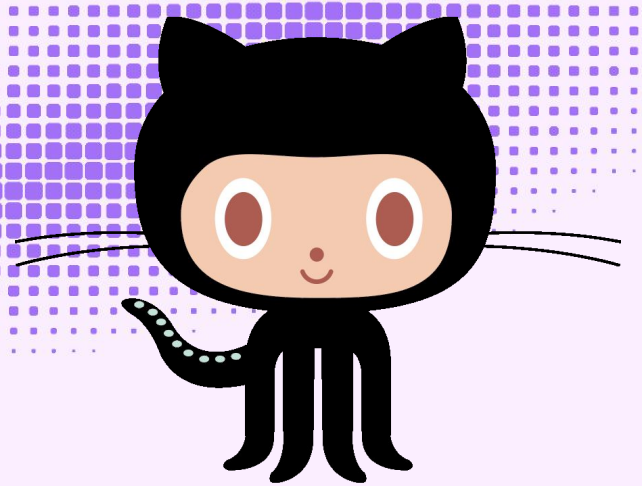
- .env should be "untracked" and will be added if we run the `git add .` command.

- Navigate to the **.gitignore** file inside the codebase.

- Add ".env" under the last comment, and save!

Concept:

`git status` should return .env in red to indicate that it is untracked / modified. Git status displays the current state of your working directory and staging area



EXERCISE 2

Env Files & Branches

- Run `git status` again, .env should vanish! And your .gitignore should be modified
- To prepare for the next exercise each partner follow only their own specific instructions. **Very Important!**

Partner A

- Run:

```
git add .

git commit -m”fix: add .env”

git push
```

Partner B

- **After A Pushes Run:**

```
git fetch origin

git reset --hard origin/main

git clean -fd
```

BREAK TIME

Any Questions?



Done!



Done!



Exercise 3

Feature branches & merging Pull Requests



Exercise 4

Triggering and resolving merge conflicts

EXERCISE 3

Branching & PRs

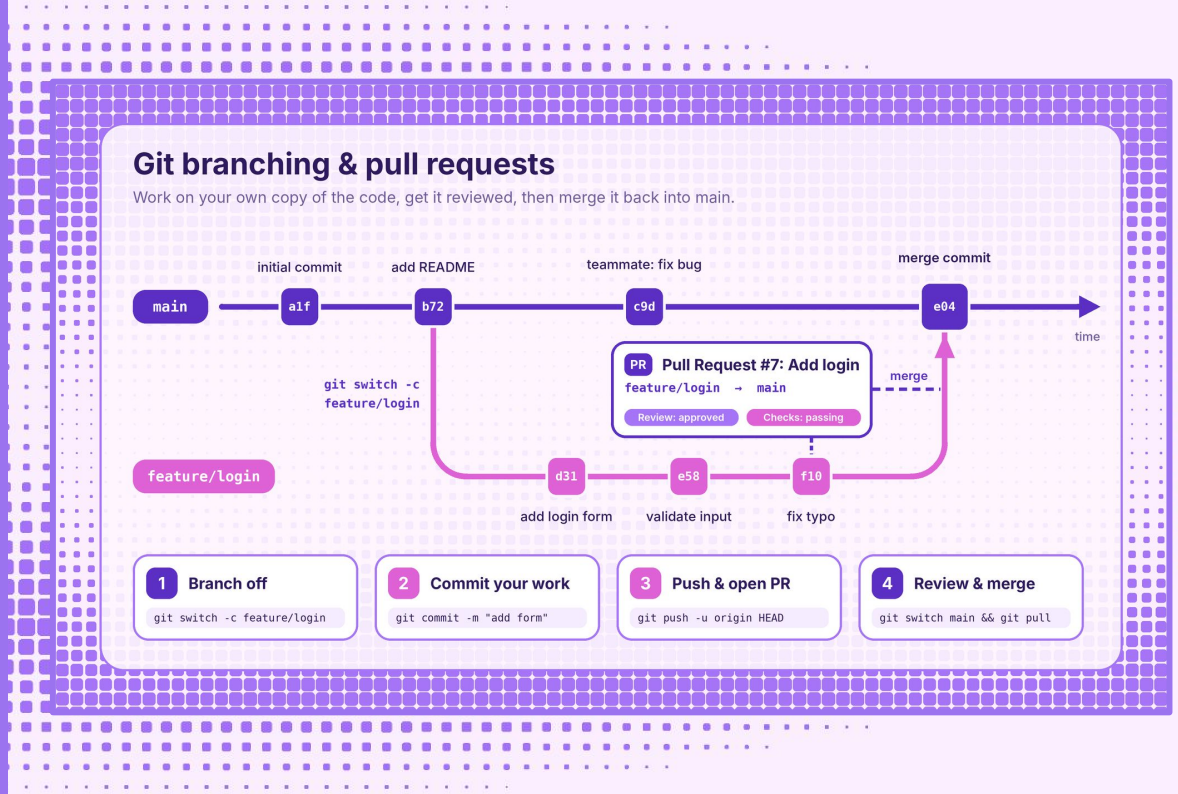
- Both partners create your own branch with:

`git switch -c <yourname>`

ex. `git switch -c cameron`

- What is a branch and why do we need or use them?

- Branches help us separate new features that might not be stable from the main branch. It is important in collaboration.



EXERCISE 3

Branching & PRs

- Two files exist for each partner: `partner_a.md`, and `partner_b.md`.
- **Both partners, fill in your name to the file assigned to you. And run the following:**

Partner A

- Run:

```
git add partner_a.md

git commit -m ”add my name”

git push -u origin <yourname>
```

Partner B

- Run:

```
git add partner_b.md

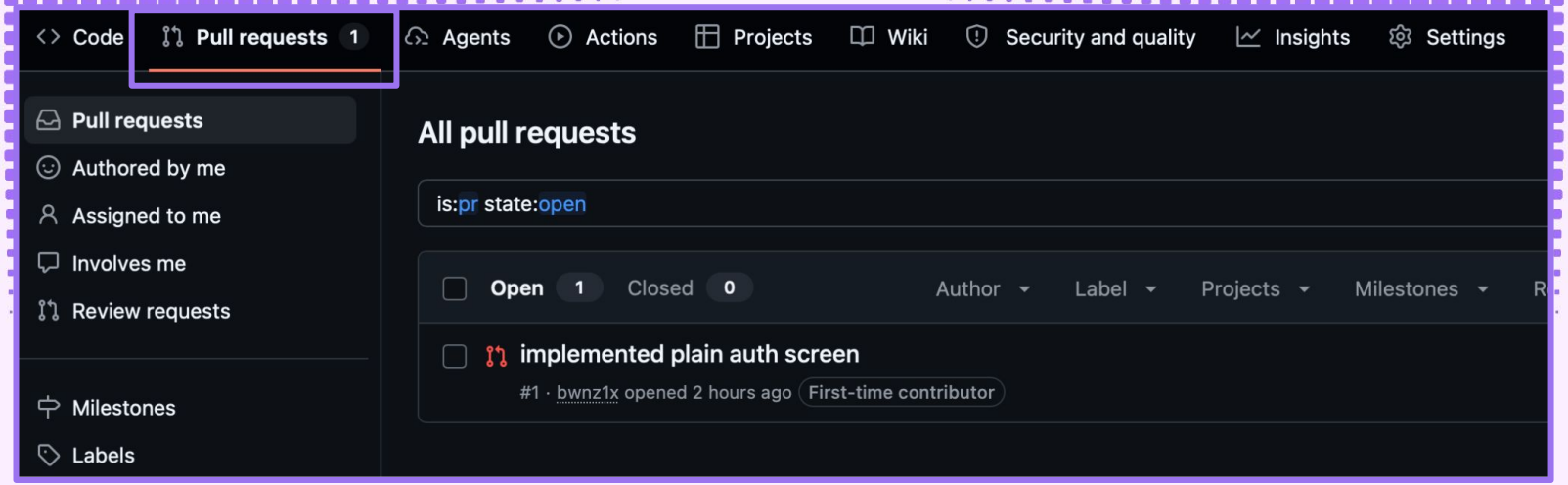
git commit -m”add my name”

git push -u origin <yourname>
```

EXERCISE 3

Branching & PRs

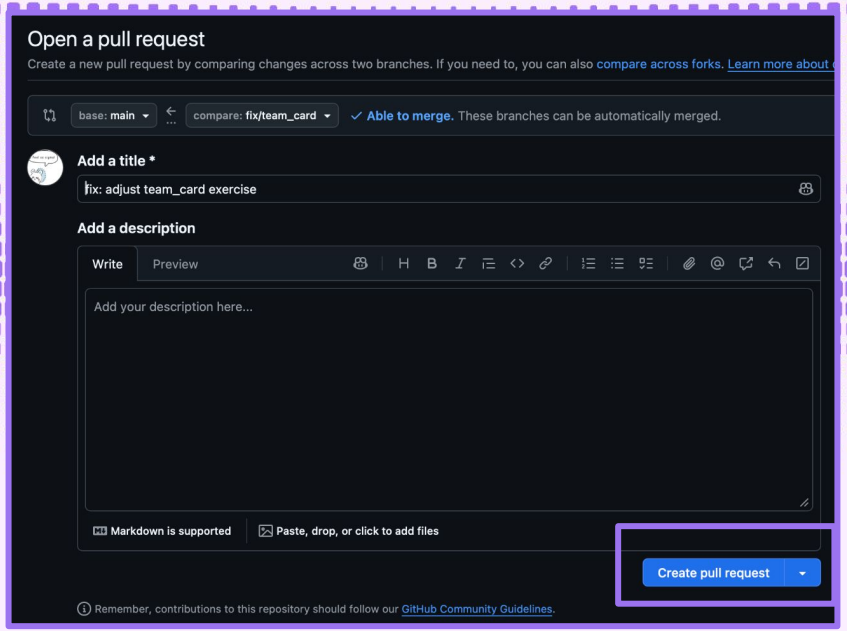
- Both partners navigate to partner A's repository on GitHub
- **Both partners may follow:** The Compare & Pull Request button. But it only does the latest, so visit the Pull Requests tab to do the one you made.



EXERCISE 3

Branching & PRs

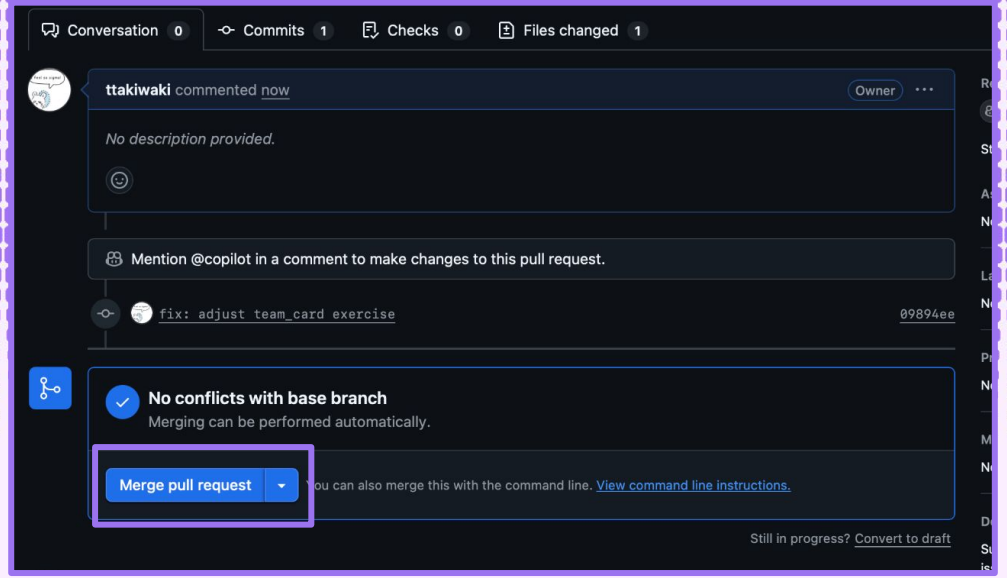
- **Both partners may follow:** Press the Create Pull Request button.



EXERCISE 3

Branching & PRs

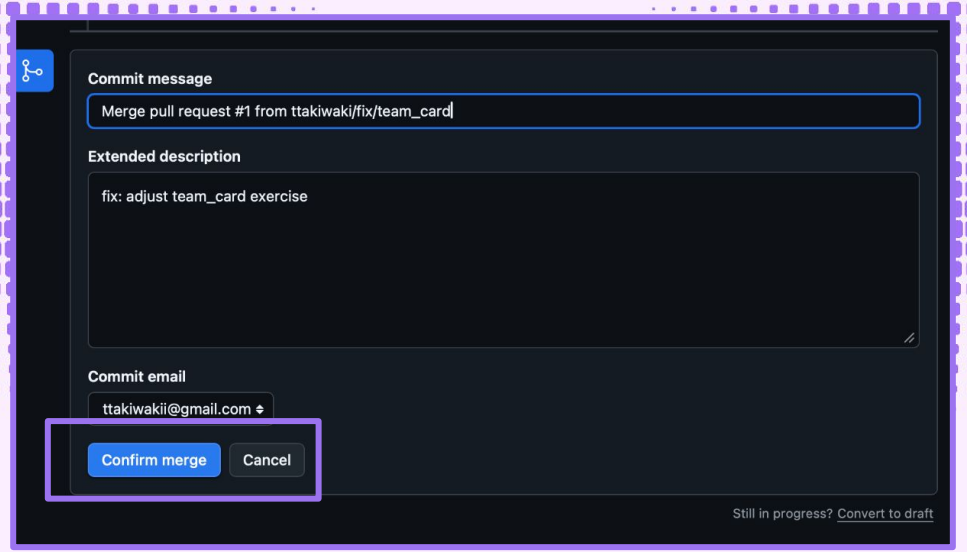
- **Both partners may follow:** Press the Merge Pull Request button.



EXERCISE 3

Branching & PRs

- **Both partners may follow:** Press the Confirm Merge button.



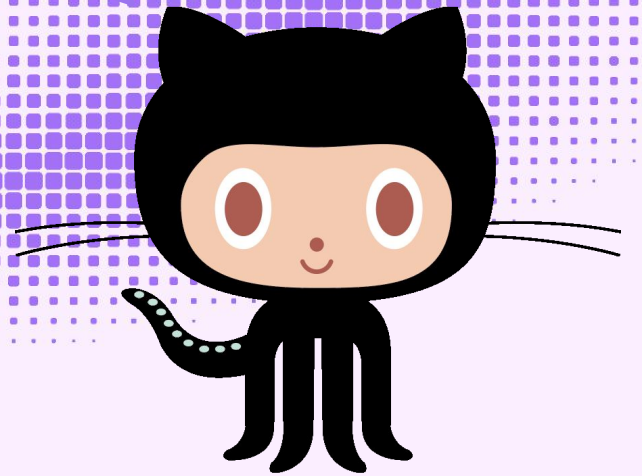
EXERCISE 3

Branching & PRs

- What just happened?
- Opening a Pull Request (PR) allowed you to propose changes from your branch so GitHub could review the code before touching main. Once GitHub verified the diff and confirmed there were no collisions with your partner's updates, clicking **Merge** cleanly pulled your commits into the repository.

Concept:

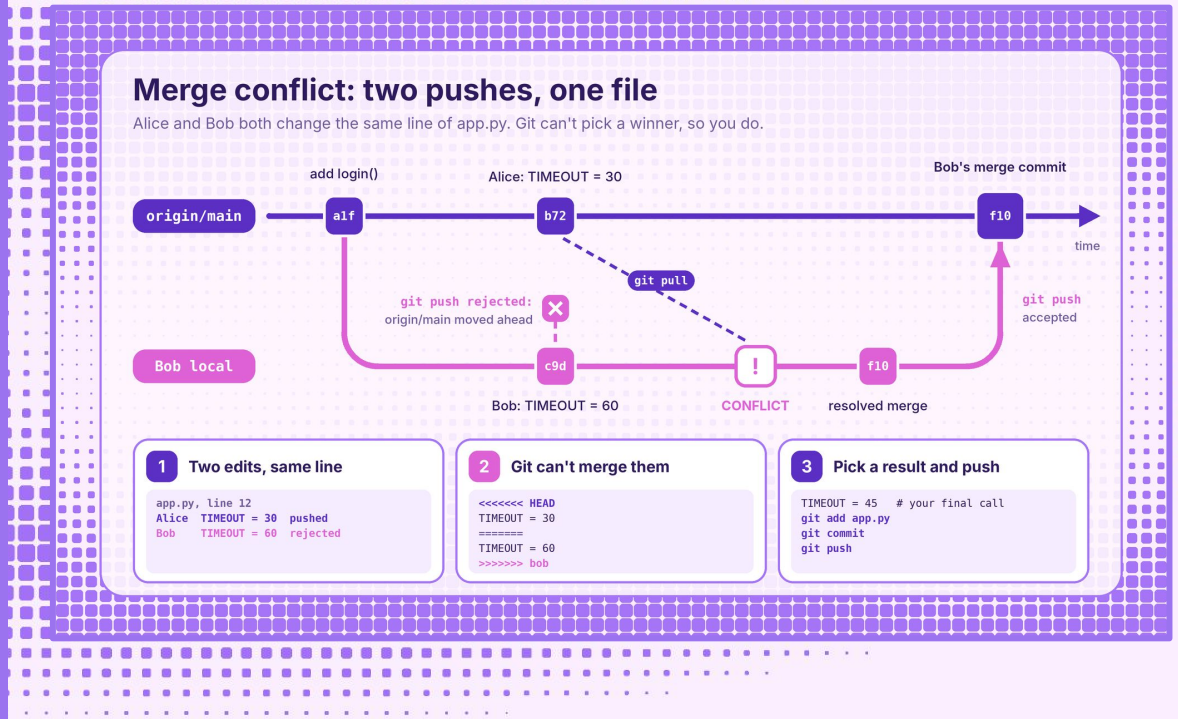
A **Pull Request (PR)** is a proposal to merge changes from your branch into the main branch. It gives your team a place to review the code diff, discuss updates, and catch bugs before any changes become permanent.



EXERCISE 4

Surviving Merge Conflicts

- What is a merge conflict?
- A **merge conflict** happens when Git cannot automatically combine changes because two people edited the exact same line in the exact same file.
- Unlike our PR exercise: where conflicts were avoided because each partner edited completely different files (partner_a.md vs. partner_b.md)



EXERCISE 4

Surviving Merge Conflicts

- Run:

```
git switch main

git pull
```

- Both partners open `team_card.md`, And replace the placeholder with your name. Additionally **without telling your other partner**, fill in a team name. **Keep it a secret from them!**

Partner A Example:

```
1  > **WARNING:** Do not edit Lines 6, 7, 8 until Phase 4 (Merge Conflicts)!
2  > When instructed, BOTH partners will edit the lines they own at the exact same time!
3
4  ## Members
5
6  - **Partner A:** Cameron
7  - **Partner B:** [Replace with Partner B Name]
8  - **Team Name:** Fall Hacks 2026
```

Partner B Example:

```
1  > **WARNING:** Do not edit Lines 6, 7, 8 until Phase 4 (Merge Conflicts)!
2  > When instructed, BOTH partners will edit the lines they own at the exact same time!
3
4  ## Members
5
6  - **Partner A:** [Replace with Partner A Name]
7  - **Partner B:** Gleb
8  - **Team Name:** Fall Hackers
```

EXERCISE 4

Surviving Merge Conflicts

- Partner A will do their steps first, and Partner B must wait until A is done before proceeding.

Partner A

- Run:

```
git add team_card.md

git commit -m”update team name”

git push
```

Partner B

- Run: `(Wait until A is done…)`

```
git add team_card.md

git commit -m”update team name”

git push
```

EXERCISE 4

Surviving Merge Conflicts

- Partner B should have encountered an error. Git attempts to **merge** the remote branch with B's local. Since the file `team_card.md` was changed on both, Git stops the merge and asks for your confirmation.

- Partner B Run:

```
git pull
```



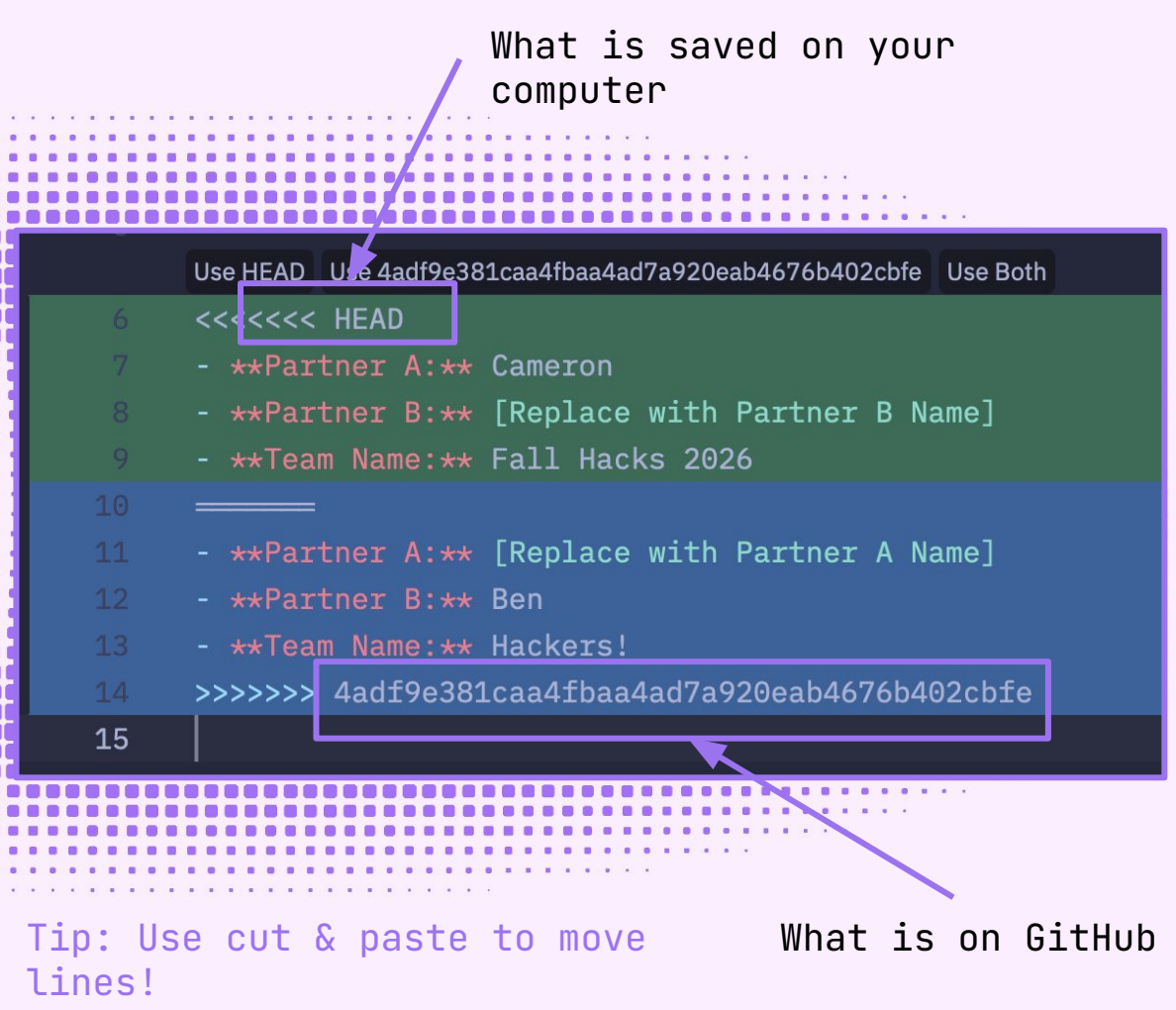
EXERCISE 4

Surviving Merge Conflicts

- Partner B should have something like the big image:

- A & B, work together to try and resolve it so it looks clean like the bottom one. Remove all the arrows and equals. Agree on a name!

```
- **Partner A:** Cameron
- **Partner B:** Ben
- **Team Name:** Fall Hacks 2026
```



EXERCISE 4

Surviving Merge Conflicts

- Once you've agreed on a file that you're ready to push, Partner B Run:

```
git add team_card.md

git commit -m "we did it"

git push
```

- Now check GitHub to see that both changes exist and the team name is there!

> **WARNING:** Do not edit Lines 6, 7, 8 until Phase 4 (Merge Conflicts)! the exact same time!

Members

- **Partner A:** Cameron
- **Partner B:** Ben
- **Team Name:** Fall Hacks 2026

WRAP UP

Main Takeaways

Exercise 1

The Core Loop: git add, git commit, git push is your everyday workflow. You can initialize locally, or on GitHub. Use git clone, or set your remote url.

Exercise 2

Protect your secrets: Always use .gitignore to block .env files and keep API keys off public GitHub repos.

Exercise 3

Branch & Collaborate: Isolate new features on separate branches, open Pull Requests, and review code safely with your team.

Exercise 4

Conflicts Aren't Scary: Merge conflicts just mean Git needs your help choosing which lines to keep—read the markers (<<<<<<<) and resolve them together.

WRAP UP

Any Questions?






Done!	Done!	Done!	Done!

Thank You!

Good Luck at **Fall Hacks!**

